

28

Sentence Transformers

Turning whole sentences into meaning-vectors

Compare and search text by meaning, not just words

AI/ML Engineer Learning Series · Deep Dive Edition

What it does	Turns a sentence into a meaningful vector
Vector =	A list of numbers capturing meaning
Key power	Similar meanings get similar vectors
Used for	Semantic search, RAG, clustering, matching
Your project	The retrieval engine in your RAG chatbot

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What are Embeddings?	Meaning as numbers
2	Sentence Transformers	Whole sentences, not just words
3	Meaning Lives in Space	Similar text sits close together
4	Measuring Similarity	Cosine similarity made simple
5	Semantic Search	Search by meaning, not keywords
6	Using It in Code	A real sentence-transformers example
7	The RAG Connection	How your chatbot finds answers
8	Picking a Model	Good defaults to know
9	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What are Embeddings?

An embedding is a list of numbers that captures the meaning of something. You met the idea briefly in the Transformers topic: words become vectors of numbers. The clever part is that these numbers are arranged so that similar meanings get similar numbers.

So the word 'king' and 'queen' would have embeddings that are close together, because they're related in meaning. 'King' and 'banana' would be far apart. The embedding turns fuzzy human meaning into precise numbers a computer can compare and search.

■ *Embeddings are one of the most important ideas in modern AI. They're the bridge that lets computers work with meaning instead of just exact text matching. This topic and the next few all build on this.*

2. Sentence Transformers

Regular embeddings work on single words. But often you want the meaning of a WHOLE sentence or paragraph. Sentence Transformers are special models that turn an entire sentence into one meaningful vector — capturing what the whole sentence is about, not just individual words.

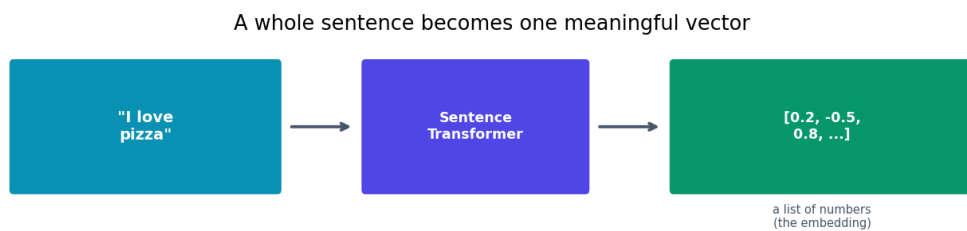


Fig 1 — A Sentence Transformer turns a whole sentence into one vector that captures its overall meaning.

This matters because meaning comes from words working together. 'The movie was not bad at all' is positive, even though 'bad' appears. A sentence transformer understands the whole thing, producing a vector that reflects the true overall meaning. This is exactly what powers the retrieval in your RAG chatbot.

3. Meaning Lives in Space

Here's the beautiful idea. Each sentence becomes a vector, and you can imagine these vectors as points in space. Sentences with similar meanings end up close together; sentences about different things end up far apart. Meaning becomes distance.

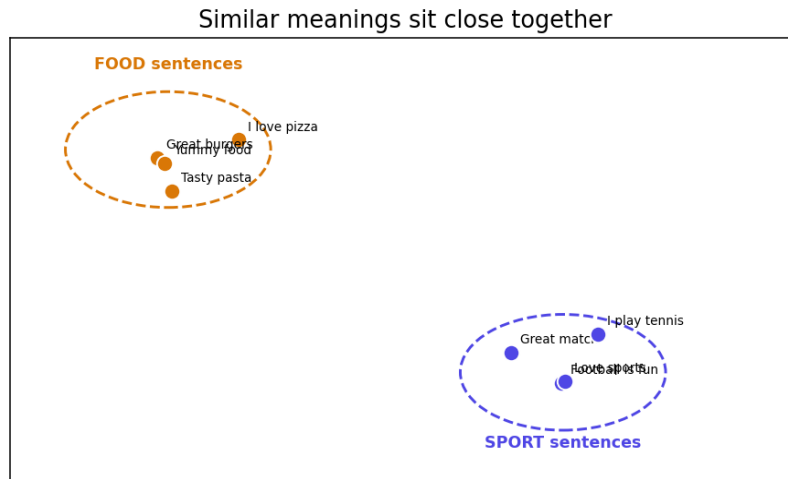


Fig 2 — Sentences about food cluster together; sentences about sport cluster elsewhere. Similar meaning = close together.

In this picture, all the food sentences gather in one area, and all the sport sentences gather in another — automatically, just from their meaning. The model was never told these categories. It learned to place similar meanings near each other. This is what makes searching and grouping by meaning possible.

4. Measuring Similarity

Once two sentences are vectors, how do we measure how similar they are? The standard tool is cosine similarity. It looks at the ANGLE between two vectors. A small angle means they point the same way — similar meaning. A large angle means different meaning.

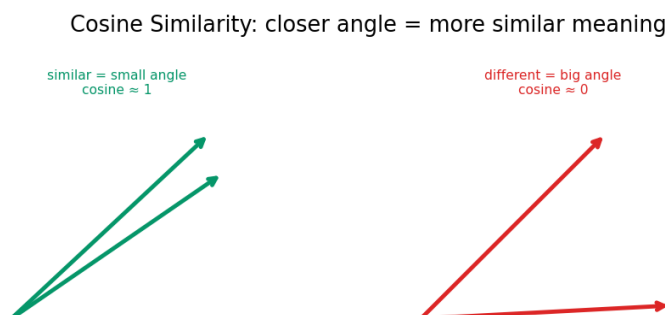


Fig 3 — Cosine similarity measures the angle between vectors. Smaller angle = more similar meaning.

Cosine similarity gives a score from -1 to 1 (usually 0 to 1 for text). A score near 1 means 'almost the same meaning', near 0 means 'unrelated'. This single number lets you rank how well any two pieces of text match — the foundation of semantic search.

■ *Why the angle and not plain distance? Because it focuses on direction (meaning) rather than length. Two sentences can have the same meaning even if one is longer; cosine similarity handles that nicely.*

5. Semantic Search

This is the killer use. Old keyword search only finds exact word matches. If you search 'cheap places to eat' but a document says 'affordable restaurants', keyword search misses it — no shared words. Semantic search finds it, because the MEANINGS match.

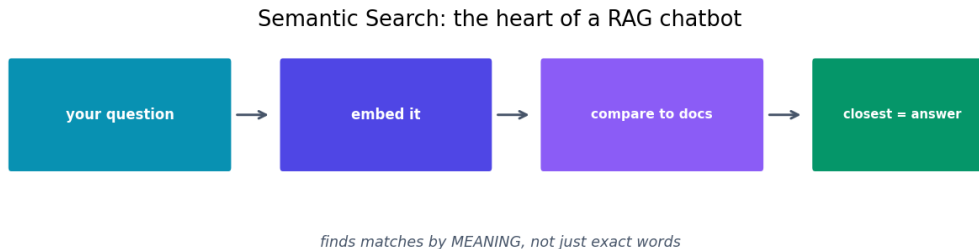


Fig 4 — Semantic search: embed the question, compare to documents by meaning, return the closest matches.

The process is simple: turn your question into a vector, turn all your documents into vectors, then find the document vectors closest to the question vector (by cosine similarity). The closest ones are the most relevant answers — even if they share no exact words. This is the heart of how your RAG chatbot finds the right context to answer a question.

6. Using It in Code

The sentence-transformers library makes this remarkably easy — a few lines to embed text and compare it:

```
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer, util

# Load a small, fast, popular model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Turn sentences into vectors
sentences = ['I love pizza', 'Pasta is delicious', 'I play football']
embeddings = model.encode(sentences)
print(embeddings.shape)  # (3, 384) -> 3 vectors of 384 numbers

# Compare similarity between two sentences
sim = util.cos_sim(embeddings[0], embeddings[1])
print(sim)  # high -> both about food

sim2 = util.cos_sim(embeddings[0], embeddings[2])
print(sim2)  # low -> food vs football
```

■ The model 'all-MiniLM-L6-v2' is a fantastic default: small, fast, and good quality. It's one of the most popular sentence embedding models and a great choice for most projects, including RAG.

7. The RAG Connection

Your PDF RAG Chatbot uses sentence transformers as its retrieval engine. Here's how the pieces fit, which the next few topics expand on:

- **Step 1:** Split your PDF into chunks of text.
- **Step 2:** Use a sentence transformer to embed every chunk into a vector.
- **Step 3:** Store all those vectors (in FAISS — next topic).
- **Step 4:** When a user asks a question, embed the question too.
- **Step 5:** Find the chunks whose vectors are closest to the question's vector.
- **Step 6:** Feed those relevant chunks to an LLM to write the final answer.

So the sentence transformer is what lets your chatbot understand a question's MEANING and find the right part of the PDF — even when the user's words don't exactly match the document's words. It's the matchmaker between question and answer.

■ *Your RAG chatbot combined this with BM25 (a keyword method) in a hybrid retrieval setup. That mixes meaning-based search (sentence transformers) with exact-word search (BM25) for the best of both — a smart, modern design.*

8. Picking a Model

Model	Good for
all-MiniLM-L6-v2	Fast, small, great all-round default
all-mpnet-base-v2	Higher quality, a bit slower
multi-qa-MiniLM	Tuned for question-answer search
paraphrase-multilingual	Many languages at once

Start with all-MiniLM-L6-v2 — it's fast and good enough for most things. If you need higher accuracy and can spare the speed, all-mpnet-base-v2 is a strong upgrade. For non-English text, pick a multilingual model. As always, the Hugging Face hub lists many more with their benchmarks.

9. Common Mistakes

Mistake	Why it's a problem	Fix
Using word embeddings for sentences	Loses whole-sentence meaning	Use a sentence transformer
Different model for query vs docs	Vectors don't match	Use the SAME model for both
Plain distance not cosine	Length skews results	Use cosine similarity
Embedding huge texts at once	Loses detail, hits limits	Split into smaller chunks
Re-embedding every search	Slow	Embed docs once, store the vectors
Expecting keyword matching	It matches meaning, not words	That's the point — use it for meaning

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
Embedding	numbers capturing meaning
Sentence transformer	embeds a whole sentence
Similar meaning	vectors close together
Cosine similarity	angle between vectors (0 to 1)
Semantic search	find by meaning, not keywords
Install	<code>pip install sentence-transformers</code>
Load model	<code>SentenceTransformer('all-MiniLM-L6-v2')</code>
Embed text	<code>model.encode(sentences)</code>
Compare	<code>util.cos_sim(vec1, vec2)</code>
Good default model	<code>all-MiniLM-L6-v2</code>
Higher quality	<code>all-mpnet-base-v2</code>
Powers	semantic search, RAG, clustering

The one idea to remember: sentence transformers turn whole sentences into vectors where similar meanings sit close together. Compare them with cosine similarity to search and match text by meaning — the foundation of semantic search and RAG.

Next Topic: LLMs (Large Language Models) — the giant models behind ChatGPT and Claude. We'll see what they really are, how they work, and how to use them well.